**ПРАКТИЧЕСКАЯ РАБОТА №3**

**ФОРМАТЫ ДАННЫХ**

**Выполнил:** Моисеев Андрей
**Группа:** 23-ИС

---

# 1. Ответы на теоретические вопросы

**Чем отличаются текстовые и бинарные форматы?**

Текстовые форматы (JSON, XML, YAML) представляют данные в виде символов, понятных человеку. Их удобно читать, редактировать вручную и отлаживать, но они избыточны по размеру и медленнее обрабатываются. Бинарные форматы (Protobuf, MessagePack, CBOR) кодируют данные в последовательность байтов: они компактнее и быстрее, но нечитаемы без специальных инструментов и требуют программной обработки.

**Что такое схемная эволюция?**

Схемная эволюция — это способность формата добавлять или удалять поля без нарушения работы старых систем. Благодаря ей новый код может читать данные, созданные старой версией программы, а старый код — корректно обрабатывать данные с новыми полями (неизвестные поля пропускаются, отсутствующие получают значения по умолчанию). Это обеспечивает обратную и прямую совместимость.

---

# 2. Результаты выполнения шагов

## 2.1. Таблица 1. Сравнение текстовых форматов

| Формат | Размер (байт) | Читаемость (1–5) | Есть ли комментарии? |
|---|---|---|---|
| JSON | 211 | 5 | Нет (стандарт не поддерживает) |
| XML | 286 | 4 | Да (`<!-- -->`) |
| YAML | 168 | 5 | Да (`#`) |

Примечание: размеры получены при запуске скриптов `step_json.py`, `step_xml.py`, `step_yaml.py` на исходном профиле пользователя.

## 2.2. Таблица 2. Сводная таблица всех форматов

Заполнена по выводу скрипта run_all.py.

| Формат | Размер (байт) | Читаем ли человеком? | Нужна ли схема? |
|---|---|---|---|
| JSON | 211 | Да | Нет |
| XML | 286 | Да | Нет |
| YAML | 168 | Да | Нет |
| MessagePack | 118 | Нет | Нет |
| CBOR | 121 | Нет | Нет |
| Protobuf | 90 | Нет | Да |

---

# 3. Выводы

**1. Почему Protobuf и MessagePack оказались меньше JSON?**

Protobuf не хранит имена полей: вместо них используются числовые теги, заданные в схеме, а значения кодируются в компактном бинарном виде (например, целые числа — переменной длины). MessagePack также кодирует структуру бинарными маркерами типов, не повторяя имена ключей в текстовом виде. JSON же хранит все ключи как строки в кодировке UTF-8 и добавляет пробелы и отступы, что заметно увеличивает размер сообщения.

**2. В каком формате данные было легче всего читать вручную?**

Легче всего данные читались в YAML — он минималистичен, не содержит лишних скобок и кавычек, а структура задаётся отступами. JSON также читаем, но избыточен из-за скобок и кавычек. XML оказался наиболее громоздким и наименее удобным для восприятия человеком.

**3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?**

В JSON нет формальной схемы и числовых тегов полей — структура определяется именами ключей «на лету», типы не контролируются. При удалении или переименовании поля старый код может получить неверные данные или ошибку. В Protobuf каждое поле имеет уникальный номер-тег и строгий тип, заданные в файле .proto. При чтении неизвестные теги пропускаются, а отсутствующие поля

получают значения по умолчанию, поэтому старый и новый код остаются совместимыми. Именно это и обеспечивает схемную эволюцию.